Ca' Foscari University of Venice

Development Methodology

Matteo Zanioli

matteo.zanioli@unive.it

Ca' Foscari University of Venice

Good organization - reasons

- Enhanced productivity: A well-organized team can better allocate resources, assign tasks, and manage time. This helps in minimizing wasted effort, allowing developers to focus on the right priorities and reducing bottlenecks in the workflow.
- Clear Roles and Responsibilities: When everyone understands their role and responsibilities, there's less confusion about who is doing what. This clarity helps in preventing overlap, reducing duplicated efforts, and ensuring that all aspects of a project are covered.
- Improved Communication: Organization fosters clear communication channels within the team. It helps in setting up regular meetings, feedback loops, and documentation practices, which are essential for keeping everyone aligned and informed about project progress and potential issues.
- Better Problem Solving and Decision Making: A structured team can more efficiently identify challenges and work together to find solutions. With defined processes in place, decisions can be made more quickly and effectively, reducing delays in development.
- Scalability and Adaptability: As projects grow or change, a well-organized team can adapt more easily. It can handle increased complexity, onboard new team members smoothly, and shift focus to meet evolving business needs without disrupting the workflow.
- Higher Quality and Consistency: Organization ensures that development standards, coding practices, and testing processes are consistently followed. This results in higher-quality code, fewer bugs, and a more stable product.

Ca' Foscari University of Venice

Structured teams

- Clear Hierarchy: Structured teams have a defined hierarchy and clear reporting lines. Roles and responsibilities are well-defined, and there is often a chain of command that must be followed for decisions.
- Predictability: These teams follow established processes and procedures, which makes their work more predictable. Planning is done in advance, and milestones are clearly set.
- Consistency and Stability: Due to their defined processes, structured teams tend to deliver consistent results. This stability is especially beneficial for long-term projects where maintaining a steady pace is important.
- Specialization: Team members often have specialized roles, such as a dedicated front-end developer, back-end developer, or tester. This allows each person to focus deeply on their area of expertise.
- Less Adaptability: While structured teams can be very efficient for well-defined projects, they may struggle to adapt quickly to sudden changes or unexpected requirements, as decision-making can be slower and adjustments need to go through the established hierarchy.

Ca' Foscari University of Venice

Flexible teams

- Flat Structure: Flexible teams tend to have a flatter organizational structure, with less emphasis on hierarchy. Decision-making is often more democratic, and team members may have more autonomy.
- Agility and Adaptability: These teams can quickly pivot and adapt to changing requirements or new priorities. This makes them suitable for dynamic environments where client needs or project scopes frequently shift.
- Cross-functional Skills: In flexible teams, members often take on multiple roles or work across different areas of expertise. This cross-functionality allows for more fluid collaboration and faster problem-solving.
- Innovation and Creativity: Because flexible teams are less constrained by rigid processes, they often foster a more creative environment. This can lead to innovative solutions and a willingness to experiment with new ideas.
- Risk of Inconsistency: Without clear guidelines and processes, flexible teams can sometimes lack consistency in their output. There is a risk of varying quality if there is no standardization or if roles become too blurred.

Ca' Foscari University of Venice

Cross-functional teams

- Diverse Skill Sets: Cross-functional teams consist of members with different skills and expertise, such as developers, designers, testers, and product managers. This variety allows the team to handle all aspects of a project within a single group.
- End-to-End Responsibility: These teams are often responsible for delivering complete features or products, from concept to delivery. They have the capability to work on design, development, testing, and deployment, without needing to rely heavily on external teams.
- Faster Iteration and Collaboration: With all necessary skills in one place, cross-functional teams can collaborate more effectively and iterate quickly. They can make decisions together, reducing delays that come from waiting on other teams.
- Greater Flexibility: These teams can adapt to changing project requirements more easily because they can shift focus or redistribute tasks internally. This makes them ideal for agile methodologies where frequent adjustments are common.
- Potential Coordination Challenges: Since members come from different backgrounds, cross-functional teams may need time to align their perspectives and communication styles, which can sometimes slow down decision-making initially.

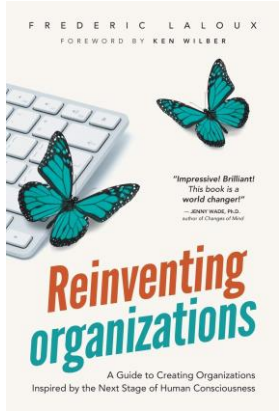
Specialized teams

- Focused Expertise: Specialized teams consist of members who all have expertise in a specific area, such as a team of front-end developers, a team of back-end developers, or a dedicated QA team. Each team focuses deeply on its own domain.
- Efficiency in Specific Tasks: Because each team is specialized, they can often perform tasks related to their domain more efficiently, leveraging deep expertise and refined processes to achieve high-quality results.
- Clear Division of Responsibilities: Roles and tasks are clearly defined in specialized teams, which reduces overlap and ensures that specific tasks are handled by those best qualified for them. This can simplify management and task allocation.
- Dependency on Other Teams: Specialized teams often rely on other teams for different parts of a project. For example, a front-end team may need to wait for the back-end team to complete their work. This dependency can lead to slower progress if coordination is not well managed.
- Less Adaptability: Since each team focuses on a specific area, they may be less adaptable to changes that require a broader range of skills. Adjustments in project scope might require more coordination between multiple specialized teams.

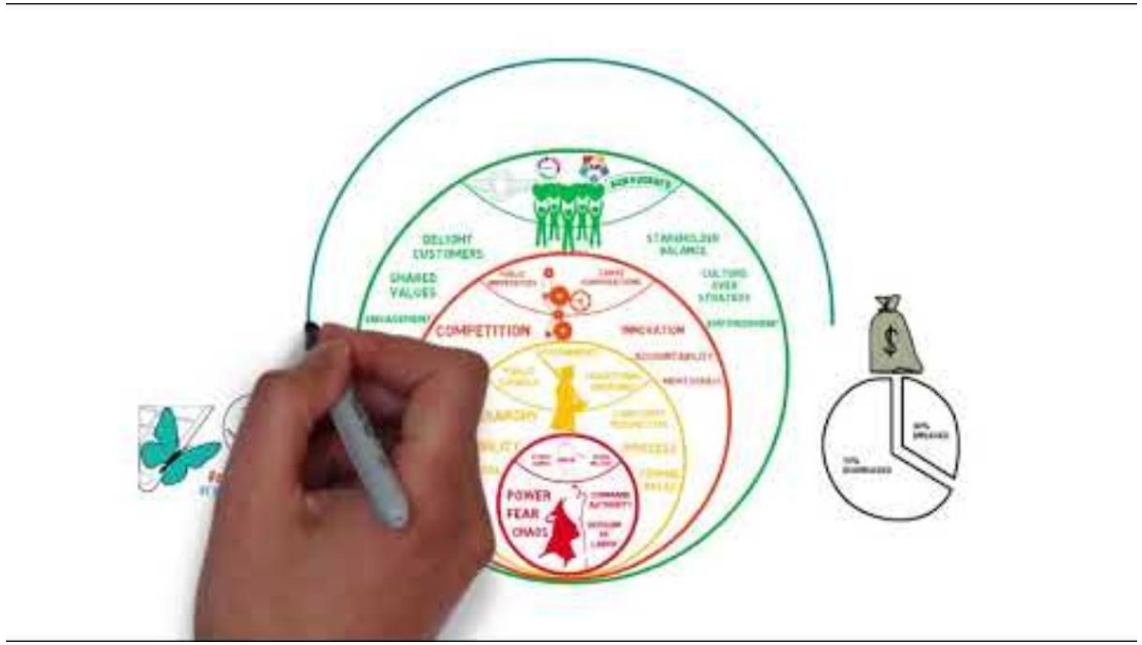
Ca' Foscari University of Venice

Organizations models – zoom out

- Complex topic and out of scope for this course.
- Different models could be applied and implemented.
- Company organization, obviously has an impact on the team organization.



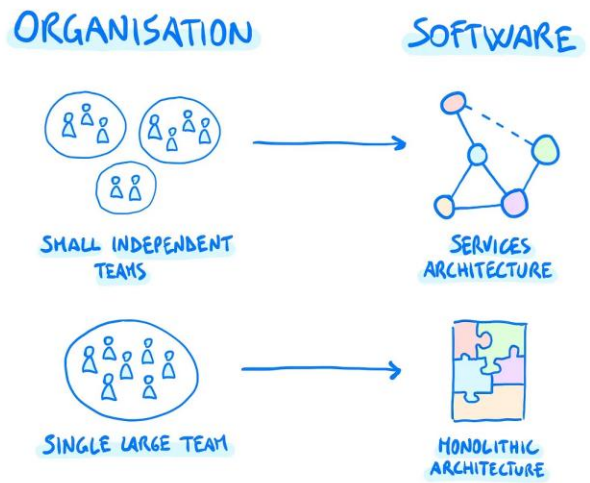
https://tinyurl.com/28nsk3dl



Development methodology

- Based on development methodology used, the structure and communication style of a team can change significantly.
- Or, generally speaking team structure shapes how communication happens, which, in turn, shapes how software gets made.

- Conway's law: "*Any organization that designs a system (defined broadly) will produce a design whose structure is a copy of the organization's communication structure.*" – *Melvin Conway (1967)*



source: https://refactoring.fm/p/team-topologies

Agile Methodology

- Team Structure: Agile teams are typically cross-functional and self-organizing, meaning that they include members from different disciplines (e.g., developers, testers, designers) and have the autonomy to make decisions without needing to go through a rigid hierarchy.
- Communication Style: Agile emphasizes frequent communication and collaboration, often through daily stand-up meetings, sprint planning, and retrospectives. This ensures that everyone stays aligned with project goals and can quickly adapt to changes.
- Flexibility: The structure of Agile teams allows for quick adaptation to evolving requirements, as they work in short iterations (sprints) and adjust priorities frequently based on feedback.

Scrum

- Team Structure: In Scrum, teams have defined roles like Scrum Master, Product Owner, and Development Team. The Scrum Master focuses on facilitating processes, while the Product Owner manages the product backlog and prioritizes tasks.
- Communication Style: Scrum relies heavily on structured communication through ceremonies like daily stand-ups, sprint reviews, and retrospectives. These ceremonies foster transparency, feedback, and continuous improvement.
- Focus on Goals: The team's communication revolves around the sprint goal, which helps keep the team focused on delivering a specific set of features or improvements within a given time frame.

Kanban

- Team Structure: Kanban teams can be more fluid and role-flexible compared to Scrum teams. There may not be a defined role like a Scrum Master, but the focus is on managing workflow using a Kanban board.
- Communication Style: Communication is often visual and asynchronous, centered around the Kanban board, which displays the status of tasks and helps the team manage work-in-progress limits. This allows team members to see progress and bottlenecks at a glance.
- Continuous Improvement: Kanban encourages incremental change, so communication tends to focus on optimizing workflows and removing bottlenecks, rather than formalized meetings.

Waterfall

- Team Structure: Waterfall teams are often specialized and hierarchical, with a clear separation of roles and stages of development (e.g., requirements gathering, design, development, testing). Each stage is completed before moving to the next.
- Communication Style: Communication in Waterfall is typically document-centric and top-down, with formal requirements, specifications, and design documents that are reviewed at each stage. Feedback loops are longer, as changes are generally made after the testing phase.
- Less Flexibility: Waterfall's structured approach makes it harder for the team to adapt to changes once a phase is completed. Communication focuses on progress tracking and ensuring that each phase meets predefined criteria before moving forward.

Devops

- Team Structure: DevOps encourages a collaborative culture between development and operations teams, often blurring the lines between the two. The goal is to promote continuous integration, delivery, and deployment.
- Communication Style: Communication is highly collaborative and often involves real-time feedback through shared tools (e.g., CI/CD pipelines, monitoring dashboards). This enables rapid identification and resolution of issues.
- Focus on Automation: Automation tools and scripts often facilitate communication by providing updates, alerts, and logs that are shared across the team, reducing the need for manual status updates.

Ca' Foscari University of Venice

What makes a team organization good?

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
GCCLXM

Ca' Foscari University of Venice

What makes a team organization good?

- Not a unique answer is available: depends on context, people, and many other factors.
- I believe that the teal organizational model can maximize the value generated by people and thus foster the development of an effective team.
- Therefore, I agree with those who identify the following themes as key to a successful organization: autonomy, cognitive load, distributed responsibilities (product, tech and people)
- "Team Topologies", written by Matthew Skelton and Manuel Pais in 2019 provide an answer: **it's all about cognitive load**.

Cognitive Load

- There is wide consensus in our industry around splitting engineering orgs into small teams.
- There is less consensus on how to do so.
- A good team should own an amount of software that can fit in their head (as a team).
- Cognitive load is the amount of mental effort that is required in your working memory to perform your job.

- Three kinds of cognitive load:
  - Intrinsic: your tech skills, which are intrinsic to your work (e.g., how classes are defined in Java)
  - Extraneous: stuff you have to remember that does not bring value, so it is mostly a distraction (e.g., how do I deploy this app, again?)
  - Germane: knowledge about your business domain and the problems you need to solve (e.g., what are good abstractions for financial data, or best practice for an e-commerce checkout)
- Germane load is the only knowledge that creates actual value: teams have to maximized germane and minimized intrinsic and extraneous cognitive load.

source: https://refactoring.fm/p/team-topologies

Reducing Intrinsic Load

- It is about technology.
- Good tech choices that make stuff easy to build.
- You can also work on team capabilities:
  - training
  - pair programming
  - hiring

Reducing Extrinsic Load

- It is about bureaucracy.
- Focus on good developer experience.
- Optimizing for fast and frequent releases, good tooling, and a culture of continuous delivery.

Optimizing Germane Load

- Organization should enable team to apply their domain knowledge to solve problems of the right size, minimizing the amount of plumbing and non-business-related effort.
- Splitting teams to make them own specific parts of the product lets them laser focus their domain knowledge and improve the quality.

How do you organize teams?

- Four main types of teams:
  - Stream-aligned teams: teams who own and work independently on a slice of the business domain. Also called "product teams".
  - Enabling teams: it's about developing skills, they research new best practice or new tech and are consulted to improve the output of other teams. Final goals is to transfer the knowledge acquired.
  - Complicated-subsystem teams: teams that work on niche problems, that require specific skills, that would otherwise bottleneck the work of other teams. Generally, they are occasional involved in projects.
  - Platform teams: teams that build infrastructure that is used by other teams and that enables their work (e.g., CI/CD, tooling and any product/framework born for internal use).

Ca' Foscari University of Venice

Friendship in the workplace

1 Go to wooclap.com

2 Enter the event code in the top banner

Event code
UOLGJR

Ca' Foscari University of Venice

Friendship in the workplace

Positive aspects

- **Increased Job Satisfaction**: Studies suggest that employees who have close friendships at work tend to report higher levels of job satisfaction. Friendship fosters a sense of belonging, which enhances morale and emotional well-being.
- **Improved Collaboration**: Friendships can lead to better teamwork. When colleagues trust and support one another, communication tends to flow more easily, and conflicts are often resolved more quickly. This can increase overall productivity.
- **Higher Employee Engagement**: Research, including Gallup studies, shows that having a "best friend" at work is strongly correlated with higher employee engagement. Engaged employees are more motivated, produce better outcomes, and are less likely to leave the company.
- **Support Networks**: Friends at work can provide emotional support during stressful times, reducing burnout and improving resilience. This can contribute to a healthier work environment.

Negative aspects

- **Blurring Boundaries**: When friendships become too personal, it can be hard to separate professional and personal life. This might lead to biased decision-making, such as favoritism or conflicts of interest.
- **Conflict Risk**: Close friendships can sometimes make it difficult to provide or accept constructive criticism, which can hinder performance. If disagreements arise, they may have a greater emotional impact compared to conflicts with non-friend colleagues.
- **Exclusivity and Cliques**: While friendships can strengthen bonds within small groups, they can also unintentionally exclude others, leading to cliques. This can harm team cohesion and foster a divisive atmosphere in the workplace.
- **Work-Life Balance**: Close friendships at work can sometimes lead to an overlap between work and personal time, making it harder to maintain boundaries and disconnect after work hours.

Failure

- Napster
  - Shawn Fanning and Sean Parker
  - Their friendship helped launch Napster and brought them together as passionate innovators
  - Legal issues and conflicts with the music industry overwhelmed the company (shut down in 2001)
  - Friendship was not enough

- Fab.com
  - a design-focused e-commerce startup founded by Jason Goldberg and Bradford Shellhammer, who were close friends; over $1B at its peak
  - As the company grew, the friendship between Goldberg and Shellhammer started to fray, with disagreements over the company's vision and direction
  - Fab.com suffered from mismanagement, overspending, and poor strategic decisions, eventually leading to its collapse and sale for a fraction of its peak valuation.

Success

- Google
  - Larry Page e Sergey Brin
  - friends and PhD students at Stanford
  - Google was founded in 1998
  - Despite the immense pressures of managing one of the world's largest tech companies, Page and Brin maintained a strong relationship

- Microsoft
  - Bill Gates and Paul Allen
  - Childhood friends with a shared passion for computers
  - Microsoft was founded in 1975
  - Allen stepped down from active management in 1983 due to health reasons

Ca' Foscari University of Venice

Is your team a family or a sports team?

1. Go to wooclap.com
2. Enter the event code in the top banner

Event code
JLSORX

Ca' Foscari University of Venice

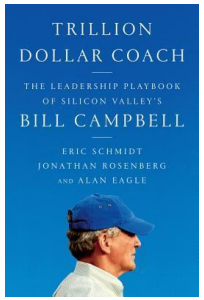
Is your team a family or a sports team?

- Reed Hastings, former CEO at Netflix, says that family is about unconditional support, while professional players on sports teams are judged on results, and there is no shame in being let go or changing jerseys every now and then.

https://tinyurl.com/27hqqqwe

- Bill Campbell, coach career, treat people on his team like close family
- He broke down the walls between human and professional personas and cared about teammates as human beings.
- He argued that people who feel appreciated end up performing at their very best.

https://tinyurl.com/2bkbwa4s



Ca' Foscari University of Venice

Pyramid of Motivation

- **Psychological safety**: the foundation of any motivated team is people who feel safe to take risks, voice their opinions, and fail without fear of repercussions. Actions:
  - Encourage people to give feedback
  - Hold regular retrospectives
  - Push people a tiny bit beyond their comfort zone
- **Good communication**: transparency (share everything), frequency (share information regularly), bi-directionality (take in consideration team's input and show how it influence decisions)
- **Alignment**: two versions of alignment that you should take care of:
  - Alignment of actions to company goals
  - Alignment of company goals to personal goals
- **Autonomy**: focus on delegation and expanding people's scope defining boundaries, delegate meaningful responsibility and encourage initiative

Ca' Foscari University of Venice

Collaboration Tools

- Project management and issue tracking
  - Jira, https://www.atlassian.com/software/jira
  - Trello, https://trello.com/
  - Asana, https://asana.com/
  - Basecamp, https://basecamp.com/
- Communication tools
  - Microsoft Teams, https://www.microsoft.com/en-us/microsoft-teams
  - Slack, https://slack.com/
  - Discord, https://discord.com/

Prioritization

MoSCoW method

- Definition: The MoSCoW method is a prioritization technique based on their importance.
- M: Must have – non-negotiabile items
- S: Should have – important items
- C: Could have – desiderable but not essential
- W: Won't have (this time) – out of scope

Eisenhower Matrix

- Definition: a.k.a. Urgent-Important Matrix, a time management tool that helps *individual prioritize* tasks based on their urgency and importance
- Q1: Urgent and important (do first)
- Q2: Not Urgent but Important (plan)
- Q3: Urgent but not important (delegate)
- Q4: Not urgent and not important (eliminate)

Backlog

- **Definition**: The backlog is a prioritized list of work items to be completed, commonly used in Agile or Scrum methodologies. It includes all the features, bug fixes, improvements, and ideas that the team wants to implement in the product.
- **Types of Backlog**:
  - Product Backlog: This is a high-level list of all the tasks, features, and enhancements needed to develop the product. It's owned by the Product Owner and is continuously refined (groomed) as priorities evolve.
  - Sprint Backlog: In Scrum, the sprint backlog is a subset of the product backlog selected for a specific sprint (time-boxed iteration). It represents the tasks the team commits to completing during that sprint.
- **Backlog Refinement**: Also known as backlog grooming, this is the ongoing process of adding detail, estimates, and priorities to items in the product backlog. The aim is to ensure that the most important tasks are ready to be tackled in the next sprint.
- **Prioritization**: It's crucial to prioritize the backlog based on factors such as business value, customer needs, and technical feasibility. Methods like MoSCoW or weighted scoring can help in prioritizing items effectively.
- **Granularity**: Items in the backlog are often written as user stories (in Agile) and vary in detail. Higher-priority items are more detailed and broken down, while lower-priority items can be broader and less defined.

Roadmap

- **Definition**: A roadmap is a high-level strategic plan that outlines the direction and goals of the product over time. It gives a long-term vision and communicates what the team plans to deliver in future releases.
- **Purpose**: The roadmap provides a strategic overview of product development, aligning stakeholders (product management, engineering, marketing, etc.) on the priorities and timeline. It helps in setting expectations with stakeholders and provides transparency on the product's evolution.
- **Types of Roadmaps**:
  - Product Roadmap: This focuses on the product's evolution, outlining planned features, milestones, and timelines for delivery. It's generally divided into short-term (immediate), mid-term, and long-term goals
  - .Technology Roadmap: A roadmap that highlights key technological developments, infrastructure upgrades, or architectural changes planned for the future.
- **Flexibility**: Unlike fixed project plans, roadmaps in Agile are flexible and iterative. They are updated regularly to reflect changes in business priorities, customer feedback, or market conditions.
- **Visual Representation**: Roadmaps are often visualized using timelines or feature lists, which help stakeholders quickly grasp the direction of the product. Tools like Gantt charts or Kanban boards can be used to represent roadmaps.

Backlog and Roadmap

- The roadmap outlines the strategic vision, while the backlog contains the actionable items that lead to fulfilling that vision. The product backlog translates the roadmap's high-level goals into tasks that the development team can work on.
- The roadmap sets priorities at a strategic level, and the backlog refines these priorities into concrete work items. The roadmap helps guide backlog prioritization, ensuring that the most important and valuable tasks are tackled first.

Backlog and Roadmap - challenges

- Balancing Short-Term and Long-Term Goals: Teams need to balance working on immediate tasks (from the backlog) with keeping an eye on the long-term vision (the roadmap).
- Overcrowded Backlog: An overly large backlog can become unmanageable. Regular grooming sessions are necessary to keep it relevant.
- Roadmap Misalignment: Sometimes the roadmap becomes disconnected from the day-to-day tasks in the backlog, leading to misalignment between strategic goals and operational work.

Backlog and Roadmap – best practice

- Keep Backlog Prioritized and Refined: Regularly update and prioritize the backlog based on current business needs and feedback.
- Ensure Roadmap Flexibility: Be prepared to adjust the roadmap based on new information, such as market changes or customer insights.
- Collaborative Management: Both the backlog and the roadmap should be managed collaboratively, involving stakeholders from different departments to ensure alignment.

Task assignment

- Definition: Task assignment involves allocating specific tasks or work items to team members based on their skills, availability, and the project's needs.
- Key consideration
  - Skill Matching: Ensure tasks are assigned to team members who have the right skill set and expertise to complete the work effectively. This may involve balancing experienced members with those who are still learning.
  - Workload Balance: Avoid overloading any individual. Even distribution of tasks ensures that no one is overwhelmed, which helps maintain productivity and morale.
  - Task Clarity: Ensure tasks are well-defined, with clear requirements and objectives. Ambiguities can lead to confusion, delays, and errors.
  - Ownership: Assigning ownership of tasks is important to create accountability. Each team member should clearly understand what they are responsible for.
  - Autonomy: Empower team members to self-organize and choose tasks, particularly in Agile teams, where this autonomy can drive motivation and innovation.
- Challenges
  - Skill Gaps: Sometimes, team members lack the necessary skills for certain tasks, requiring training or reassignment.
  - Team Collaboration: Complex tasks may require collaboration between multiple team members, which can lead to coordination challenges.
  - Task Reallocation: As priorities shift, tasks may need to be reassigned. Flexibility is essential in a dynamic development environment.

Ca' Foscari University of Venice

Resources management

- Definition: Resource management refers to the effective planning, allocation, and tracking of resources (e.g., people, time, equipment) required to complete tasks and meet project objectives.
- Key resources
  - Human Resources: The team members available to complete tasks. This includes managing their time, skills, and workload.
  - Time: Managing deadlines and ensuring that tasks are completed within the planned timeline. Time is one of the most critical resources in project management.
  - Budget: Allocating and managing financial resources to ensure the project stays within budget.
  - Tools and Equipment: Ensuring the team has access to the necessary tools, software, and infrastructure to complete tasks efficiently.
- Principles
  - Capacity Planning: This involves assessing how much work a team can take on within a given period. It helps prevent over-commitment.
  - Resource Allocation: Distribute resources based on priorities, ensuring that high-priority projects or tasks are adequately supported. This may involve assigning team members, budgeting, or providing necessary tools.
  - Resource Optimization: Ensure that resources are used efficiently and avoid under-utilization. This can involve reassigning resources to where they are needed most.
  - Monitoring and Adjustment: Continuously monitor resource usage and make adjustments when necessary to ensure that projects remain on track.
- Challenges
  - Scarcity of Resources: There may be more tasks than available resources, leading to bottlenecks and delays. Prioritization becomes critical in such scenarios.
  - Resource Conflicts: In multi-project environments, team members may be pulled in different directions, leading to conflicts in priorities.
  - Unforeseen Changes: Sudden changes in project scope, team availability, or technical requirements can disrupt resource planning.

Ca' Foscari University of Venice

Goal setting framework

MBO – Management By Objectives

- Performance management system focus on setting specific and measurable goals
- Top-down approach, where management sets the objectives
- Long term timeframe (annual)

OKR – Objectives and Key Results

- A goal setting framework, focus on setting ambitious goals and defining measurable outcomes (Key Results)
- Collaborative approach, involves both top-down and bottom-up input
- Short term timeframe (quarterly)